

Multiple Neural Networks and Weighted Voting

Ethem Alpaydin

Department of Computer Engineering, Bogazici University
TR-80815 Bebek Istanbul Turkey

Abstract

We propose to train a number of neural networks (nets) independently, either with different learning algorithms or with the same algorithm but with different sets of parameters and then take a vote over them in the form of a weighted majority. All the nets learn essentially the same task but converge to different solutions due to different learning parameter values, e.g. net structure. Voting schemes investigated are static vs. dynamic, where in dynamic voting scheme, the network complexity is also taken into account. The system assigns a weight of "confidence" to each participating net proportional to the net's success and complexity. As empirical work shows, having multiple nets significantly improves generalization, i.e., higher success is achieved in classifying previously unseen data. This framework is not limited to neural nets but can be applied to learning systems in general.

1. Introduction

One major problem with using an artificial neural net to handle a certain task is that, associated with each learning method is a set of parameters whose values when changed lead to different percentages of success. There are no clear rules by which the optimal values can be computed; they are tuned by trial and error. Furthermore, one may choose one of several learning algorithms for the same task each having different advantages and constraints, and one frequently is unable to pinpoint the most suitable. In this paper, we propose to train a number of neural nets for the same task independently and take a vote over their responses. We have used two completely different learning methods: a nearest-neighbor based method named GAL and a discriminant based method; a feed-forward net trained with back-propagation. This both significantly improves generalization to previously unseen data and also, with such an approach, the designer no longer needs to guess the optimal values of parameters.

2. Multi-GAL

Grow and Learn (GAL) is a variant of the nearest neighbor search method where one stores only a subset of the training set and not the whole [1]. Starting from nothing, a training pattern is stored if the net's prediction of that pattern is wrong. The net's prediction is the label of

the closest stored pattern using a suitable metric. Two or three sweeps over the training set are enough to get no prediction error on the training set where finally the stored patterns are those that lie closest to class boundaries. However the same training set when ordered in different ways leads to different subsets being stored with a significant variance in success on previously unseen test set although all of them classify the training set with no error. One may order the training set in different ways and train a number of GAL nets and then take a vote over them [2]. Two possible voting schemes are *simple* and *weighted*. In simple voting, one just counts how many of the nets vote for a class and then chooses the class having the maximum count. In weighted voting, a net together with the class code, also indicates its "certainty" on its output which is used to weigh the net's vote. In the case of GAL, one computes the distance to the two closest patterns. If these two have the same label, the net is very certain; if they have different labels, the certainty decreases as the input gets closer to their median; the certainty is the difference of the two distances.

No of nets	No of Units	Success	Voting type
NN	600.0	93.2	-
1	118.4	90.2	-
3	247.5	92.3	s
3	247.5	93.4	w
5	321.8	93.3	s
5	321.8	94.2	w
7	371.7	94.6	w
12	447.3	94.9	w

Table 1. Simulation results with multi-GAL.. Reported are averages of 30 runs, and are on test data unseen during training.

Simulation results. A study on the recognition of handwritten numerals has shown this approach to lead to better generalization (Table 1). Not only average success on test set increases but also success variance on different runs decreases (fig. 1). Another interesting point is that although individual nets all stored a *subset* of the training data, the overall net had a success higher than that of the nearest neighbor (NN) where *all the patterns* are stored. In the simulation reported here, input patterns are 16 by 16 bitmaps belonging to 10 classes. Training and test sets

contain 600 patterns each. Weighted voting seems to perform better than simple voting. As the number of participating nets increase, so do average success and number of units stored. Depending on the application and

the implementation constraints, the optimal number of nets may be determined which trades off the cost of additional nets with the improvement in success they bring.

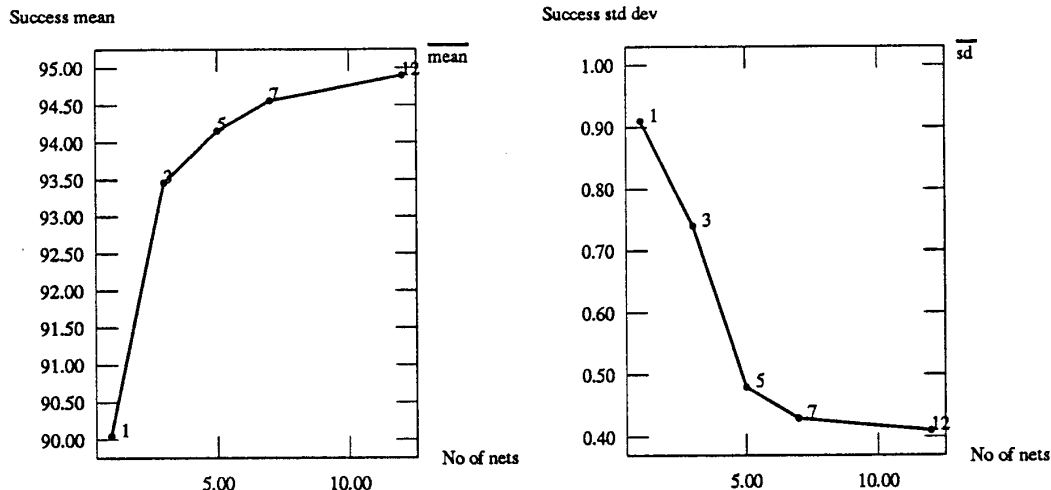


Figure 1. Mean and standard deviation of success for various net sizes with multi-GAL using weighted voting.

3. Multi-backpropagation

Perhaps the most significant problem with the back-propagation algorithm is that, given a certain training set or application, there are no rules to compute the optimal net architecture, i.e., the number of hidden layers and hidden units, and their receptive and projective field sizes and shapes. It has been proposed to use: (i) one's knowledge of the application and/or the learning algorithm [4] to pre-structure the net before training [8], (ii) to extend algorithms so that they can also dynamically modify the net structure during learning by adding/removing units/connections [2], (iii) to use genetic algorithms where better net architectures are produced from less fit parent nets [6].

Nets with more modifiable connections converge faster and perform better on the training set but may risk to overfit and not generalize well to unseen test data. Smaller nets find simpler solutions and simpler solutions are more probable [11]. The usual approach is trial and error, i.e., starting from a simple net, one increases complexity step by step until increase in success (by cross-validation) is no longer worth the additional complexity. Instead of looking for one net with the optimal architecture, one may decide to have a number of nets with different architectures and take a vote over them. A number of nets are chosen with different number of hidden layers and units and are trained separately on the same training data.

One possible weighting scheme is as used with GAL, namely by having weights equal to certainties of nets where certainty is the difference in activation levels of the two

most active class units. This we name *static voting*. Another possible scheme is *dynamic voting* where these weights are modifiable and are modified as a function of the utility of the net. Success increases the utility of a net but as we prefer simpler solutions, bigger nets are less useful. Utility then can be used as weights in a weighted-majority type of voting scheme. These weights are initially all equal and are modified during training after each iteration. Whenever the overall response is correct, the weight of the net L who voted correctly is updated as follows:

$$W_L = W_L + \text{reward} / \text{no_correct} - \text{nc}_L / \mu * \text{no_correct} * \text{max_nc}$$

reward is a value that is initially small and is slowly decreased. Decrementing value added to W by a factor of nc - the number of connections - has the effect that bigger nets are rewarded less. We assume that complexity increases linearly with the number of connections. no_correct is the number of nets who voted correctly. If less nets voted correctly, they are rewarded more. max_nc is the maximum number of connections, i.e., size of the biggest used net. By choosing μ greater than 1.0, one rewards even the nets who has the maximum number of connections, although of course in very small amount. After each update, weights are normalized to a constant sum of 1.0 to prevent them from growing without bounds. In the general case, the number of connections need to be replaced by the number of modifiable free parameters or the "description length." Thus extended, participants of the voting may be very different learning methods. Theoretical work done on a similar weighted voting scheme where the description length is not taken into account is reported [10].

Simulation results. A 2D hypothetical signal is taken and 6 back-propagation nets are trained on the same data using randomly sampled 1% of the signal (Fig. 3). Test set contains 7074 patterns and there are 6 classes. All nets have one hidden layer where the number of hidden units are

2, 4, 8, 16, 32, and 64. By exponentially increasing the number of hidden units, one favors simpler solutions and also covers the whole possible range with few nets. Simulation results are given in table 2 and figures 2 and 3.

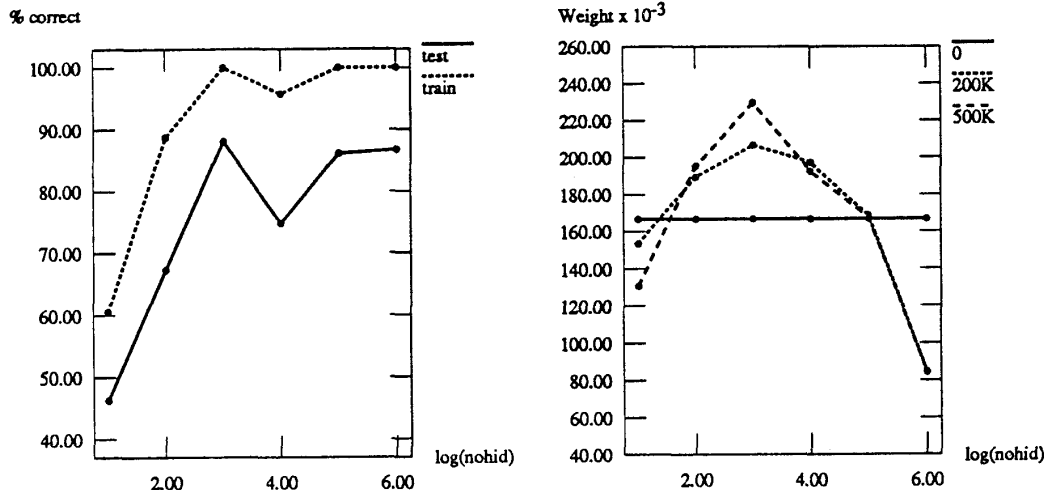


Figure 2. To the left, percent success in classifying the training and test sets after 500,000 iterations for nets of different sizes. X axis is the logarithm of the number of hidden units. To the right, evolution of weight values of different nets; initially, after 200,000 and 500,000 iterations. As seen, the net with 16 hidden units did not converge well probably due to initial weights chosen randomly, although it could have performed very well. This is exactly the point of having multiple nets and taking a vote; even if one of the nets does not perform well, others will compensate for that.

No of hidens	No of conns	Training success	Test success	W
2	16	60.6	46.3	0.13
4	32	88.7	67.3	0.19
8	64	100.0	88.1	0.23
16	128	95.8	74.8	0.19
32	256	100.0	86.1	0.17
64	512	100.0	86.7	0.08
Static	1014	100.0	91.3	
Dynamic	1014	100.0	91.5	

Table 2. Comparison of back-propagation nets of different sizes and static and dynamic voting schemes.

4. Conclusions

The result of voting is better than the best of the participants; *adding less optimal nets to the vote improves success*. W values shown in the table also indicate how useful are the participating nets. A strategy may be envisaged such that a net which is not very useful, i.e., whose weight is less than a certain threshold, may be removed to decrease computational complexity. Note that the W values have the form of a normal distribution. The optimal number of hidden units, i.e., the peak of the distribution, may be roughly computed from the existing values and a new net with that number of hidden units may

be automatically trained and added as a new participant. When a net is to be replaced by a new one, the new one if possible, may be derived from the old instead of being created from scratch. For example in the case of back-propagation, it was shown empirically that hidden units may be added one by one to an existing net and this improves the rate of convergence [3]. Thus when the number of hidden units need be increased, new units may be added and training resumed instead of training a new big net from scratch. In the opposite case, there are techniques proposed to compute quickly the relevance or importance of a unit [9]. When the number of hidden units is to be decreased, the least relevant can be removed and the rest is trained. A combination of these two strategies, namely elimination of the not-very-useful participating nets and creation of prospective useful ones may take place where in time, less optimal are exchanged for better ones. Note that the framework proposed here is different from "adaptive mixtures of local experts" [7], in the sense that here, nets do not partition the task but learn globally the same task. Work is currently going on to investigate performances on real world applications. Another interesting point to check is to use different learning, possibly non-neural, algorithms as participants in voting.

Acknowledgements. This work has started at the International Computer Science Institute, Berkeley. Thanks to Subutai Ahmad, Jerry Feldman, Franco Masulli, Steve Omohondro, Manfred Warmuth for stimulating discussions.

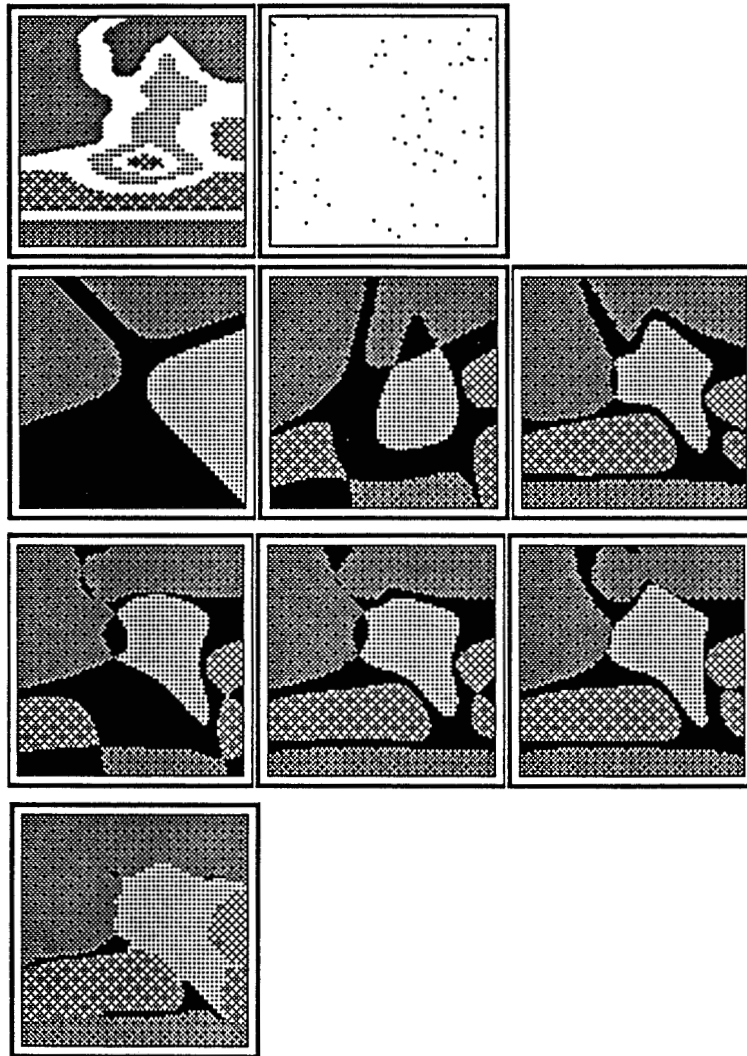


Figure 3. At top, the complete training and test data are given. The second and third lines respectively show the responses of nets with 2, 4, 8, and 16, 32, 64 hidden units after 500,000 learning iterations. The bottom line shows the result of weighted voting. Different textures stand for different classes, black points are reject cases where either none or more than one class unit is active.

References

- [1] E. Alpaydin, "Neural models of incremental supervised and unsupervised learning," *PhD thesis, EPFL*, 1990.
- [2] E. Alpaydin, "GAL: Networks that grow when they learn and shrink when they forget," *ICSI TR-91-032*, 1991.
- [3] T. Ash, "Dynamic node creation in back-propagation networks," *Connection Science*, 1, 365-375, 1989.
- [4] J. Denker et al., "Large automatic learning, rule extraction, and generalization," *Complex Systems*, vol. 1, pp. 877-922, 1987.
- [5] S.J. Hanson and L.Y. Pratt, "Comparing biases for minimal network construction with back-propagation," in *NIPS*, vol. 1, pp. 177-185, 1989.
- [6] S.A. Harp et al., "Designing application-specific neural networks using the genetic algorithm," in *NIPS*, vol. 2, pp. 447-454, 1990.
- [7] R. A. Jacobs et al. "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, pp. 79-87, 1991.
- [8] Y. le Cun et al. "Back-propagation applied to handwritten zip recognition," *Neural Computation*, vol. 1, pp. 541-551, 1989.
- [9] Y. le Cun et al. "Optimal brain damage" in *NIPS*, vol. 2, pp. 598-605, 1990.
- [10] N. Littlestone, M.K. Warmuth, "The weighted majority algorithm," *UC Santa Cruz, CRL-89-16*, 1989.
- [11] J. Rissanen, "Stochastic complexity," *Journal of Royal Stat. Soc. B*, vol. 49, pp. 223-239, 252-265, 1987.